

Лабораторная работа № 8

Централизованный мониторинг Linux-инфраструктуры с помощью Zabbix

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - готовая VM zabbix01;
 - сервер web01;
 - сервер резервного копирования backup01;
 - Zabbix Server и Frontend;
 - Zabbix Agent 2;
 - Nginx;
 - BorgBackup или подготовленный маркер успешного резервного копирования.
-

1. Цель работы

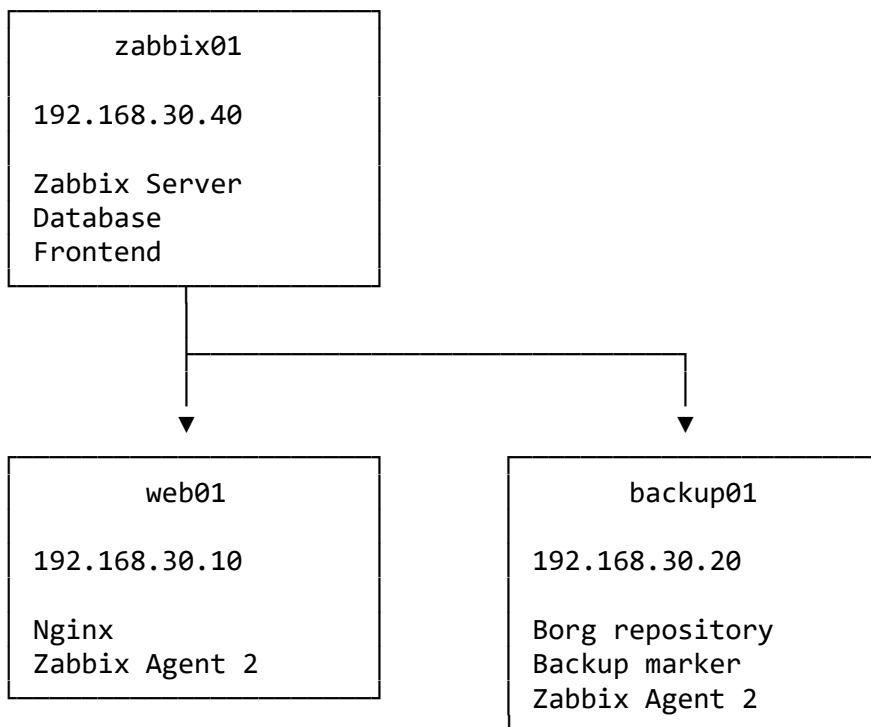
Настроить централизованный мониторинг Linux-серверов и проверить реакцию Zabbix на реальные эксплуатационные события.

В ходе работы необходимо:

- подключить Linux-серверы к Zabbix;
- настроить Zabbix Agent 2;
- добавить Hosts и Templates;
- проверить получение стандартных метрик;
- контролировать состояние службы Nginx;
- контролировать локальный порт и HTTP-ответ;
- настроить пользовательский Item через UserParameter;
- контролировать возраст последнего успешного backup;
- создать Triggers с понятными Severity и Tags;
- настроить Dashboard;
- смоделировать несколько отказов;
- проверить открытие и закрытие Problems;
- использовать Acknowledge и Maintenance.

2. Схема стенда

SERVER-LAN
192.168.30.0/24



3. Таблица адресации

Устройство	IP-адрес	Назначение
zabbix01	192.168.30.40	Zabbix Server, Database и Frontend
web01	192.168.30.10	Веб-сервер Nginx
backup01	192.168.30.20	Сервер резервного копирования

Все виртуальные машины подключаются к одной сети VirtualBox:

Internal Network: SERVER-LAN

4. Исходное состояние

Преподаватель заранее подготавливает zabbix01:

- установлен Zabbix Server;
- установлена и настроена Database;
- доступен Zabbix Frontend;
- создана административная учётная запись;
- настроено корректное время;
- разрешён TCP 10051;
- подготовлен пакетный репозиторий Zabbix Agent 2.

Адрес Frontend:

`http://192.168.30.40/zabbix`

На web01 должны быть установлены и запущены:

- Nginx;
- SSH;
- Zabbix Agent 2.

На backup01:

- SSH;
- BorgBackup;
- Zabbix Agent 2;
- каталог для маркера успешного backup.

5. Основные понятия

Объект	Назначение
Host	Наблюдаемый сервер или устройство
Host Group	Группа наблюдаемых объектов
Template	Набор типовых Items, Triggers и Graphs
Item	Одно измеряемое значение
Trigger	Условие перехода в Problem
Problem	Активное отклонение
Severity	Влияние проблемы
Tag	Метка для фильтрации и маршрутизации
Acknowledge	Подтверждение принятия проблемы в работу
Recovery	Возврат метрики в нормальное состояние
Maintenance	Режим плановых работ

6. Базовые задания

Задание 1. Проверить сетевую связность и время

На web01 выполнить:

```
hostnamectl  
ip -br addr  
ip route  
ping -c 4 192.168.30.40
```

На backup01:

```
hostnamectl  
ip -br addr  
ip route  
ping -c 4 192.168.30.40
```

На zabbix01 проверить оба сервера:

```
ping -c 4 192.168.30.10  
ping -c 4 192.168.30.20
```

На всех серверах проверить время:

```
date  
timedatectl
```

Ожидаемый результат

- все серверы доступны друг другу;
- адреса соответствуют таблице;
- системное время синхронизировано;
- расхождение времени не превышает нескольких секунд.

Некорректное время затрудняет сопоставление событий, метрик и журналов.

Задание 2. Настроить Zabbix Agent 2 на web01

Установить Agent 2 из подготовленного репозитория:

```
sudo apt update  
sudo apt install -y zabbix-agent2
```

Создать резервную копию конфигурации:

```
sudo cp -a \  
  /etc/zabbix/zabbix_agent2.conf \  
  /etc/zabbix/zabbix_agent2.conf.backup
```

Открыть файл:

```
sudo nano /etc/zabbix/zabbix_agent2.conf
```

Установить параметры:

```
Server=192.168.30.40
ServerActive=192.168.30.40
Hostname=web01
```

Проверить строки:

```
grep -E \
  '^(Server|ServerActive|Hostname)=' \
  /etc/zabbix/zabbix_agent2.conf
```

Перезапустить Agent:

```
sudo systemctl enable --now zabbix-agent2
sudo systemctl restart zabbix-agent2
```

Проверить:

```
systemctl status zabbix-agent2 --no-pager
```

Ожидаемое состояние:

```
active (running)
```

Проверить порт passive checks:

```
sudo ss -lntp | grep ':10050'
```

Проверить журнал:

```
sudo journalctl \
  -u zabbix-agent2 \
  -b \
  --no-pager
```

Задание 3. Настроить Zabbix Agent 2 на backup01

Установить Agent:

```
sudo apt update
sudo apt install -y zabbix-agent2
```

Создать резервную копию:

```
sudo cp -a \
  /etc/zabbix/zabbix_agent2.conf \
  /etc/zabbix/zabbix_agent2.conf.backup
```

Настроить:

```
Server=192.168.30.40
ServerActive=192.168.30.40
Hostname=backup01
```

Проверить:

```
grep -E \
    '^(Server|ServerActive|Hostname)=' \
    /etc/zabbix/zabbix_agent2.conf
```

Запустить:

```
sudo systemctl enable --now zabbix-agent2
sudo systemctl restart zabbix-agent2
```

Проверить:

```
systemctl is-active zabbix-agent2
sudo ss -lntp | grep ':10050'
sudo journalctl \
    -u zabbix-agent2 \
    -b \
    -n 30 \
    --no-pager
```

Важно

Значение:

Hostname=backup01

должно точно совпадать с полем **Host name** в Zabbix Frontend.

Поле **Visible name** может отличаться.

Задание 4. Добавить Hosts в Zabbix Frontend

Открыть:

Data collection → Host groups

Создать группу:

Linux Servers

Затем открыть:

Data collection → Hosts

Создать Host web01.

Параметры web01

Поле	Значение
Host name	web01
Visible name	Web Server 01
Host group	Linux Servers
Interface type	Agent
IP address	192.168.30.10
Port	10050

Связать Host с типовым Linux Template, доступным в установленной версии Zabbix.

Обычно используется шаблон, содержащий:

Linux by Zabbix agent

или:

Linux by Zabbix agent active

Создать backup01.

Поле	Значение
Host name	backup01
Visible name	Backup Server 01
Host group	Linux Servers
IP address	192.168.30.20
Port	10050

Связать тот же Linux Template.

Проверить доступность:

Monitoring → Hosts

Индикатор Agent должен перейти в доступное состояние.

Задание 5. Проверить стандартные Items

Открыть:

Monitoring → Latest data

Выбрать Host:

web01

Найти значения:

- Agent availability;
- CPU utilization;
- CPU load;
- Available memory;
- Swap usage;
- заполнение /;
- использование inode;
- входящий сетевой трафик;
- исходящий сетевой трафик;
- uptime.

Повторить для:

backup01

На самих серверах сравнить данные с командами.

CPU

uptime
nproc
top

RAM

free -h

Диск

df -hT /
df -i /

Сеть

ip -s link

Uptime

uptime -p

Вывод

Zabbix собирает измеримые значения, но их интерпретация должна учитывать:

- количество CPU;
 - baseline;
 - длительность;
 - роль сервера;
 - влияние на сервис.
-

Задание 6. Создать пользовательскую проверку службы Nginx

На web01 создать сценарий:

```
sudo nano /usr/local/sbin/check-service-zabbix.sh
```

Добавить:

```
#!/usr/bin/env bash

set -euo pipefail

service=${1:-}

case "$service" in
    nginx|ssh|zabbix-agent2)
        ;;
    *)
        printf '%s\n' \
            'ZBX_NOTSUPPORTED: unsupported service'
        exit 1
        ;;
esac

if systemctl is-active --quiet "$service"; then
    printf '1\n'
else
    printf '0\n'
fi
```

Назначить владельца и права:

```
sudo chown root:root \
    /usr/local/sbin/check-service-zabbix.sh
```

```
sudo chmod 755 \
    /usr/local/sbin/check-service-zabbix.sh
```

Проверить синтаксис:

```
sudo bash -n \
    /usr/local/sbin/check-service-zabbix.sh
```

Проверить от имени пользователя Zabbix:

```
sudo -u zabbix \
    /usr/local/sbin/check-service-zabbix.sh nginx
```

Ожидаемый результат при работающей службе:

1

Проверить неизвестную службу:

```
sudo -u zabbix \  
/usr/local/sbin/check-service-zabbix.sh unknown
```

Создание UserParameter

Создать файл:

```
sudo nano \  
/etc/zabbix/zabbix_agent2.d/service-status.conf
```

Добавить:

```
UserParameter=custom.service.status[*],/usr/local/sbin/check-service-zabbix.sh  
"$1"
```

Проверить конфигурацию Agent:

```
sudo zabbix_agent2 \  
-t 'custom.service.status[nginx]'
```

Ожидаемый результат содержит значение:

1

Перезапустить Agent:

```
sudo systemctl restart zabbix-agent2
```

Создание Item во Frontend

Открыть:

Data collection → Hosts → web01 → Items

Создать Item:

Поле	Значение
Name	Nginx service status
Type	Zabbix agent
Key	custom.service.status[nginx]
Type of information	Numeric unsigned
Update interval	30s
History	7d

Добавить Tags:

```
service=nginx  
check=systemd
```

Создание Trigger

Открыть:

Data collection → Hosts → web01 → Triggers

Создать Trigger:

Поле	Значение
Name	Nginx service is inactive on web01
Severity	High
Expression	last(/web01/custom.service.status[nginx])=0

Добавить Tags:

service=nginx
team=linux
env=lab

Задание 7. Настроить HTTP-проверку Nginx

Проверить сайт вручную с zabbix01:

```
curl -I http://192.168.30.10/
```

Ожидается:

HTTP/1.1 200 OK

Во Frontend открыть:

Data collection → Hosts → web01 → Web

Создать Web Scenario:

Поле	Значение
Name	web01 HTTP availability
Update interval	1m
Attempts	2
Agent	Zabbix

Добавить Step:

Поле	Значение
Name	Open home page
URL	http://192.168.30.10/

Поле	Значение
Required status codes	200
Timeout	5s

После сохранения Zabbix создаст связанные Web Items.

Проверить:

Monitoring → Latest data → web01

Найти данные Web Scenario:

- response code;
- response time;
- failed step.

Создать Trigger для ошибки Web Scenario.

Концептуальное условие:

failed step > 0

Название:

web01 HTTP application check failed

Severity:

High

Tags:

service=nginx
check=http
team=linux

Почему нужны разные проверки

Проверка	Что доказывает
systemd unit	Служба считается запущенной
Local port	Процесс слушает порт
Remote port	Порт доступен из сети
HTTP Scenario	Приложение возвращает ожидаемый ответ

Состояние active не доказывает, что сайт доступен пользователю.

Задание 8. Настроить контроль возраста backup

На backup01 создать каталог:

```
sudo install \  
-d \  
-m 755 \  
/var/lib/backup-monitor
```

Создать маркер успешного backup:

```
date +%s |  
sudo tee \  
/var/lib/backup-monitor/web01.last_success
```

Назначить права:

```
sudo chown root:root \  
/var/lib/backup-monitor/web01.last_success
```

```
sudo chmod 644 \  
/var/lib/backup-monitor/web01.last_success
```

Создать сценарий:

```
sudo nano \  
/usr/local/sbin/backup-age-zabbix.sh
```

Добавить:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
readonly MARKER='/var/lib/backup-monitor/web01.last_success'  
readonly UNKNOWN_AGE=999999
```

```
if [[ ! -r $MARKER ]]; then  
    printf '%s\n' "$UNKNOWN_AGE"  
    exit 0  
fi
```

```
timestamp=$(cat "$MARKER")
```

```
if [[ ! $timestamp =~ ^[0-9]+$ ]]; then  
    printf '%s\n' "$UNKNOWN_AGE"  
    exit 0  
fi
```

```
now=$(date +%s)  
age=$((now - timestamp))
```

```
if (( age < 0 )); then  
    age=0  
fi
```

```
printf '%s\n' "$age"
```

Назначить права:

```
sudo chown root:root \  
/usr/local/sbin/backup-age-zabbix.sh
```

```
sudo chmod 755 \  
/usr/local/sbin/backup-age-zabbix.sh
```

Проверить:

```
sudo bash -n \  
/usr/local/sbin/backup-age-zabbix.sh
```

Запустить от имени Agent:

```
sudo -u zabbix \  
/usr/local/sbin/backup-age-zabbix.sh
```

Ожидается небольшое число секунд.

Создание UserParameter

Создать файл:

```
sudo nano \  
/etc/zabbix/zabbix_agent2.d/backup-age.conf
```

Добавить:

```
UserParameter=custom.backup.age,/usr/local/sbin/backup-age-zabbix.sh
```

Проверить:

```
sudo zabbix_agent2 \  
-t custom.backup.age
```

Перезапустить Agent:

```
sudo systemctl restart zabbix-agent2
```

Создание Item

Bo Frontend:

Data collection → Hosts → backup01 → Items

Создать:

Поле	Значение
Name	Age of last successful web01 backup
Type	Zabbix agent

Поле	Значение
Key	custom.backup.age
Type of information	Numeric unsigned
Units	s
Update interval	1m
History	30d

Tags:

service=backup
source=web01

Создание Trigger

Для учебной работы используем RPO 24 часа:

86400 секунд

Создать Trigger:

Поле	Значение
Name	web01 backup is older than 24 hours
Severity	High
Expression	last(/backup01/custom.backup.age)>86400

Tags:

service=backup
team=linux
rpo=24h

Важно

Маркер должен обновляться только после полного успешного завершения backup job.

Сам факт появления свежего файла ещё не доказывает успешное резервное копирование.

Задание 9. Создать Dashboard

Открыть:

Dashboards

Создать Dashboard:

Linux Infrastructure

Добавить Widgets:

1. **Problems** — активные проблемы.
2. **Host availability** — доступность web01 и backup01.
3. **Top hosts** или аналогичный виджет CPU.
4. **Graph** — заполнение / на web01.
5. **Graph** — доступная память web01.
6. **Item value** — возраст последнего backup.
7. **Web monitoring** — HTTP-проверка web01.

Настроить фильтрацию по группе:

Linux Servers

Dashboard должен отвечать на вопросы:

- какие Hosts сейчас недоступны;
- какие Problems активны;
- работает ли Nginx;
- доступен ли сайт;
- растёт ли заполнение диска;
- когда выполнялся последний успешный backup.

Задание 10. Выполнить матрицу отказов

Для каждого сценария заполнить таблицу:

Сценарий	Ожидаемый Problem	Фактический результат	Recovery
Agent остановлен	Agent unavailable		
Nginx остановлен	Service inactive		
HTTP недоступен	HTTP check failed		
Backup старше RPO	Backup stale		
Maintenance active	Уведомление подавлено		

Сценарий 1. Остановить Nginx

На web01:

```
sudo systemctl stop nginx
```

Проверить:

```
systemctl is-active nginx
sudo ss -lntp | grep ':80'
curl -I http://127.0.0.1/
```


Во Frontend проверить:

Monitoring → Problems

Должны открыться проблемы:

- Nginx service is inactive;
- HTTP application check failed.

Выполнить Acknowledge:

Проблема принята в работу. Проверяется состояние Nginx.

Запустить службу:

```
sudo systemctl start nginx
```

Проверить:

```
systemctl is-active nginx  
curl -I http://127.0.0.1/
```

Во Frontend дождаться Recovery.

Сценарий 2. Остановить Zabbix Agent

На web01:

```
sudo systemctl stop zabbix-agent2
```

Проверить:

```
systemctl is-active zabbix-agent2  
sudo ss -lntp | grep ':10050'
```

Во Frontend должна появиться проблема доступности Agent или отсутствие данных.

Запустить Agent:

```
sudo systemctl start zabbix-agent2
```

Проверить:

```
systemctl is-active zabbix-agent2
```

Дождаться Recovery.

Сценарий 3. Смоделировать устаревший backup

На backup01 записать время двухдневной давности:

```
date \
--date='2 days ago' \
+%s |
sudo tee \
/var/lib/backup-monitor/web01.last_success
```

Проверить:

```
sudo -u zabbix \
/usr/local/sbin/backup-age-zabbix.sh
```

Значение должно быть больше:

86400

Во Frontend проверить открытие Problem:

web01 backup is older than 24 hours

Обновить маркер:

```
date +%s |
sudo tee \
/var/lib/backup-monitor/web01.last_success
```

Проверить:

```
sudo -u zabbix \
/usr/local/sbin/backup-age-zabbix.sh
```

Дождаться Recovery.

Сценарий 4. Использовать Maintenance

Во Frontend открыть:

Data collection → Maintenance

Создать Maintenance:

Поле	Значение
Name	web01 planned maintenance
Maintenance type	With data collection
Host	web01
Period	Текущее время, 15 минут

Во время Maintenance остановить Nginx:

```
sudo systemctl stop nginx
```

Проверить:

- метрики продолжают поступать;
- Problem может отображаться;
- уведомления должны быть подавлены, если Action настроен с учётом Maintenance.

Запустить Nginx:

```
sudo systemctl start nginx
```

Завершить или удалить тестовый Maintenance.

Вывод

Maintenance используется для плановых работ, а не для скрывтия неисправностей.

7. Диагностика Zabbix Agent

Проверка службы

```
systemctl status zabbix-agent2 --no-pager
```

Проверка порта

```
sudo ss -lntp | grep ':10050'
```

Проверка конфигурации

```
grep -E \  
'^(Server|ServerActive|Hostname)=' \  
/etc/zabbix/zabbix_agent2.conf
```

Проверка журнала

```
sudo journalctl \  
-u zabbix-agent2 \  
-b \  
-n 50 \  
--no-pager
```

Локальная проверка Item

```
sudo zabbix_agent2 \  
-t 'custom.service.status[nginx]'
```

```
sudo zabbix_agent2 \  
-t custom.backup.age
```

Проверка passive item с Zabbix Server

На zabbix01, если установлен zabbix_get:

```
zabbix_get \  
-s 192.168.30.10 \  
-k system.hostname
```

Ожидается:

web01

8. Дополнительные задания

Дополнительное задание 1. Перевести backup01 на Active Checks

На backup01 проверить:

```
ServerActive=192.168.30.40  
Hostname=backup01
```

Во Frontend убедиться, что:

Host name = backup01

Связать Host с Template для Active Agent Checks либо создать отдельный Active Item.

Для пользовательского Item выбрать Type:

Zabbix agent (active)

Перезапустить Agent:

```
sudo systemctl restart zabbix-agent2
```

Проверить журнал:

```
sudo journalctl \  
-u zabbix-agent2 \  
-b \  
-n 50 \  
--no-pager
```

Вывод

При Active Check:

- соединение иницирует Agent;
 - Agent подключается к TCP 10051;
 - совпадение Hostname и Host name обязательно;
 - входящий TCP 10050 может не требоваться для полностью активного набора Items.
-

Дополнительное задание 2. Смоделировать заполнение отдельной файловой системы

Не заполнять корневую файловую систему до 95–100%.

Добавить к web01 отдельный виртуальный диск размером 1 ГБ.

После подключения определить имя:

```
lsblk
```

Допустим, диск определился как:

```
/dev/sdb
```

Создать файловую систему:

```
sudo mkfs.ext4 /dev/sdb
```

Создать точку монтирования:

```
sudo mkdir -p /srv/monitor-test
```

Смонтировать:

```
sudo mount /dev/sdb /srv/monitor-test
```

Проверить:

```
findmnt /srv/monitor-test  
df -hT /srv/monitor-test
```

Создать тестовый файл:

```
sudo fallocate \  
-l 920M \  
/srv/monitor-test/load-test.bin
```

Проверить:

```
df -hT /srv/monitor-test
```

Во Frontend создать Item или использовать обнаруженную файловую систему.

Создать Trigger:

```
Usage of /srv/monitor-test is above 90%
```

После открытия Problem удалить файл:

```
sudo rm \  
/srv/monitor-test/load-test.bin
```

Проверить Recovery.

Дополнительное задание 3. Создать зависимость проблем

Предположим, что web01 имеет две проблемы:

- Agent unavailable;
- Nginx service inactive.

Настроить зависимость Trigger Nginx от Trigger доступности Agent.

Открыть Trigger Nginx:

Data collection → Hosts → web01 → Triggers

Добавить Dependency:

Zabbix agent is not available

Остановить Agent:

```
sudo systemctl stop zabbix-agent2
```

Ожидаемый результат

Основной проблемой отображается недоступность Agent.

Зависимая проблема службы не должна создавать лишнюю тревогу или уведомление.

Вернуть Agent:

```
sudo systemctl start zabbix-agent2
```

Вывод

Dependencies уменьшают количество вторичных Problems и помогают быстрее увидеть корневой симптом.

9. Основные команды

Agent

```
systemctl status zabbix-agent2  
sudo journalctl -u zabbix-agent2 -b  
sudo ss -lntp | grep ':10050'
```

Локальное тестирование Items

```
sudo zabbix_agent2 -t system.hostname  
sudo zabbix_agent2 -t 'custom.service.status[nginx]'  
sudo zabbix_agent2 -t custom.backup.age
```

Linux-метрики

```
uptime  
nproc  
free -h  
df -hT  
df -i  
ip -s link
```

Nginx

```
systemctl is-active nginx  
sudo ss -lntp | grep ':80'  
curl -I http://127.0.0.1/
```

Backup marker

```
cat /var/lib/backup-monitor/web01.last_success  
date +%s  
sudo -u zabbix \  
    /usr/local/sbin/backup-age-zabbix.sh
```

10. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему и таблицу адресации.
3. Настройки Agent на web01 и backup01.
4. Созданные Hosts и связанные Templates.
5. Примеры стандартных Items.
6. Конфигурацию UserParameter для Nginx.
7. Конфигурацию UserParameter для backup age.
8. Созданные Triggers и их Severity.
9. Dashboard.
10. Заполненную матрицу отказов.
11. Пример Acknowledge.
12. Результаты Problem и Recovery.
13. Краткий вывод по работе.

В отчёт не включаются:

- пароли Zabbix;
- ключи;
- Secret Macros;
- параметры доступа к Database;

- закрытые конфигурационные данные.
-

11. Чек-лист выполнения

- ☐ zabbix01 достигает наблюдаемых серверов.
 - ☐ На web01 работает Zabbix Agent 2.
 - ☐ На backup01 работает Zabbix Agent 2.
 - ☐ Hostname совпадает с Host name во Frontend.
 - ☐ Создана группа Linux Servers.
 - ☐ Созданы Hosts web01 и backup01.
 - ☐ Связан Linux Template.
 - ☐ Поступают CPU, RAM, disk и network Items.
 - ☐ Создан UserParameter состояния Nginx.
 - ☐ UserParameter проверен от имени zabbix.
 - ☐ Создан Trigger остановки Nginx.
 - ☐ Настроен HTTP Web Scenario.
 - ☐ Создан маркер успешного backup.
 - ☐ Создан Item возраста backup.
 - ☐ Создан Trigger нарушения RPO.
 - ☐ Создан Dashboard.
 - ☐ Проверена остановка Nginx.
 - ☐ Проверена остановка Agent.
 - ☐ Проверен устаревший backup.
 - ☐ Выполнен Acknowledge.
 - ☐ Для каждого Problem получен Recovery.
 - ☐ Тестовые службы и настройки возвращены в нормальное состояние.
-

12. Контрольные вопросы

1. Чем мониторинг отличается от журналирования, аудита и диагностики?
2. Какую роль выполняют Zabbix Server, Agent, Database и Frontend?
3. Чем Passive Check отличается от Active Check?
4. Почему Hostname особенно важен для Active Checks?
5. Почему состояние systemd-службы не доказывает доступность приложения?
6. Почему UserParameter должен возвращать одно машинно читаемое значение?
7. Почему свежий файл не всегда подтверждает успешный backup?
8. Для чего используются Acknowledge, Recovery, Maintenance и Dependencies?